

Koopman NMPC: Koopman-based Learning and Nonlinear Model Predictive Control of Control-affine Systems

Carl Folkestad* and Joel W. Burdick*

Abstract—Koopman-based learning methods can potentially be practical and powerful tools for dynamical robotic systems. However, common methods to construct Koopman representations seek to learn *lifted linear* models that cannot capture nonlinear actuation effects inherent in many robotic systems. This paper presents a learning and control methodology that is a first step towards overcoming this limitation. Using the Koopman canonical transform, control-affine dynamics can be expressed by a *lifted bilinear* model. The learned model is used for nonlinear model predictive control (NMPC) design where the bilinear structure can be exploited to improve computational efficiency. The benefits for control-affine dynamics compared to existing Koopman-based methods are highlighted through an example of a simulated planar quadrotor. Prediction error is greatly reduced and closed loop performance similar to NMPC with full model knowledge is achieved.

I. INTRODUCTION

Efficient control design for dynamic robotic systems is a persistent challenge when optimal task performance is sought while satisfying state, actuation, and computing constraints. Because of its ability to intuitively specify both performance metrics and constraints, model predictive control (MPC) is attractive. Advances in optimization algorithms and computing power are enabling *nonlinear* MPC (NMPC) to be deployed on robotic systems in real-time if carefully implemented [1]–[3]. One key challenge in developing NMPC, however, is to obtain a sufficiently accurate model of the system to be controlled. *Learning* algorithms can capture the salient aspects of a robot's complex mechanics and environmental interactions, thereby reducing the need for laborious system identification procedures [4]. Many approaches have been proposed to learn models intended for control design (cf. [5]–[9]). However, using these models for NMPC is not straightforward as they are either unsuitable or expensive to simulate and/or discretize to solve the resulting nonlinear program. We take a Koopman-centric approach to learn a *lifted bilinear model* of the dynamics that can be incorporated in NMPC to design close to optimal controllers that incorporate state and actuation constraints.

We are interested in learning control-affine dynamics, $\dot{x} = f(x) + g(x)u$, which allow a wide class of aerial and ground robots to be characterized. Conventionally, a control system's behavior is studied via its state space flows. In contrast, Koopman approaches study the evolution of *observables*, which are functions over the state-space. In this space, an autonomous dynamical system can be represented by a *linear* (but possibly infinite dimensional) operator [10], [11].

Data-driven methods for identifying Koopman models have received considerable attention. Dynamic Mode Decomposition (DMD) and extended DMD (EDMD) have been shown to efficiently identify finite dimensional approximations of the Koopman operator of the system dynamics [12]–[17].

Koopman-style modeling has also been extended to *controlled* nonlinear systems [9], [18]. However, most current methods approach this problem by learning a lifted *linear* model, which only allows the control vector fields, $g(x)$, to take a constant state-invariant form. This is a significant limitation, as many useful robotic systems, e.g. systems where input forces enter the system dynamics through rotation matrices, are best described by nonlinear control-affine dynamics. To overcome this limitation, we learn a model motivated by the *Koopman canonical transform* (KCT) [19], which allows a large class of nonlinear control-affine dynamical models to be *lifted* to a higher-dimensional space where the system evolution can be described by a *bilinear* (but possibly infinite dimensional) dynamical system. This lifting is achieved by careful design of the function dictionary and employing an EDMD learning method [13], [14].

Prior work on Koopman-based control design has primarily focused on applying linear MPC to lifted linear models, and has been successfully implemented in both simulated and robotic experiments [14], [20], [21]. Design with bilinear models is less explored, but connections between Koopman bilinear system descriptions and classical control concepts such as reachability and control Lyapunov functions have been presented [22], [23]. Very recently, bilinear Koopman models linearized at the current state of the system were used in MPC [24]. Another approach uses the bilinear model structure to simplify the construction of a control Lyapunov function enforced as a constraint in a nonlinear MPC method to obtain stability guarantees [25].

While a few works have addressed NMPC design for bilinear Koopman models [24], [25], little consideration has been given to practical real-time realization of these methods on robotic systems, which often require high control rates due to fast dynamics. Towards this goal, this paper presents Koopman NMPC, combining the process of learning control-affine dynamics in Koopman bilinear form with NMPC design. Our contributions are twofold. First, building on recent advances in NMPC, we develop a controller for bilinear Koopman models that uses the bilinear model structure to improve computational efficiency, making real-time computation possible. Second, we show the advantages of learning lifted bilinear models over linear models and demonstrate that the completely data-driven Koopman NMPC method can

*Both authors are with the Division of Engineering and Applied Sciences, California Institute of Technology, Pasadena, CA, USA

match the performance of a NMPC controller with full a priori model knowledge on a simulated planar quadrotor.

This manuscript is organized as follows. Preliminaries on the KCT and NMPC is presented in Section II. Then, the learning method to learn lifted bilinear models is described in Section III, and the Koopman NMPC developed in Section IV. Finally, the method is demonstrated on a simulated planar quadrotor in Section V before we conclude in Section VI.

II. PRELIMINARIES

We consider control-affine continuous-time dynamical systems of the form

$$\dot{x} = f(x) + \sum_{i=1}^m g_i(x)u_i \quad (1)$$

where $x \in \mathcal{X} \subseteq \mathbb{R}^d$, $u \in \mathcal{U} \subseteq \mathbb{R}^m$, and $f, g_i, i = 1, \dots, m$ are assumed to be Lipschitz continuous on $\mathcal{X}, \mathcal{U}$.

A. Koopman spectral theory

Before considering the effects of control inputs, we introduce the Koopman operator, which is defined for autonomous continuous-time dynamical systems:

$$\dot{x} = f_{aut}(x) \quad (2)$$

with state $x \in \mathcal{X} \subset \mathbb{R}^d$, and f is assumed to be Lipschitz continuous on $\mathcal{X}$. The flow of (2) is denoted by $S_t(x)$, defined as $\frac{d}{dt}S_t(x) = f_{aut}(S_t(x))$ for all $x \in \mathcal{X}, t \geq 0$. The *Koopman operator semi-group* $(U_t)_{t \geq 0}$, from now on simply denoted as the *Koopman operator*, is defined as

$$U_t \varphi = \varphi \circ S_t \quad (3)$$

for all $\varphi \in \mathcal{C}(\mathcal{X})$, where $\mathcal{C}(\mathcal{X})$ is the space of continuous observables $\varphi : \mathcal{X} \rightarrow \mathbb{C}$, and $\circ$ denotes function composition. Each element of $U_t : \mathcal{C}(\mathcal{X}) \rightarrow \mathcal{C}(\mathcal{X})$ is a linear operator.

An *eigenfunction* of the Koopman operator associated to an eigenvalue $\lambda \in \mathbb{C}$ is any function $\phi \in \mathcal{C}(\mathcal{X})$ that defines a coordinate evolving linearly along the flow of (2)

$$(U_t \phi)(x) = \phi(S_t(x)) = e^{\lambda t} \phi(x). \quad (4)$$

B. The Koopman canonical transform

We now return to control-affine dynamics (1) and recall when and how they can be transformed to a *bilinear* form through the *Koopman canonical transform* [19]. Let $(\lambda_i, \phi_i(x)), i = 1, \dots, n$ be eigenvalue-eigenfunction pairs of the Koopman operator associated with the autonomous dynamics of (1), $\dot{x} = f(x)$. The KCT relies on the assumption that the state vector can be described by a finite number of eigenfunctions, i.e. that $x = \sum_{i=1}^n \phi_i(x)v_i^x$ for all $x \in \mathcal{X}$, and where $v_i^x \in \mathbb{C}^d$. This is likely to hold if n is large. If not, they may be well approximated by n eigenfunctions.

When $\phi_i : \mathcal{X} \rightarrow \mathbb{R}$, the KCT is defined as

$$x = C^x z, \quad \dot{z} = Fz + \sum_{i=1}^m L_{g_i} T(x) u_i \quad (5)$$

where $z = T(x) = [\phi_1(x) \dots \phi_n(x)]^T$, $C^x = [v_1^x \dots v_n^x]$, and $F \in \mathbb{R}^{n \times n}$ is a diagonal matrix with entries $F_{i,i} = \lambda_i$.

Under certain conditions the system (5) is bilinearizable in a countable, possibly infinite basis. We restate the conditions for the existence of a bilinear form in a *finite* basis as this is of practical interest in the following theorem.

Theorem 1. [22] *Suppose there exist Koopman eigenfunctions $\phi_j, j = 1, \dots, n, n \in \mathbb{N}, n < \infty$ of the autonomous dynamics (1) whose span, $\text{span}(\phi_1, \dots, \phi_n)$, forms an invariant subspace of $L_{g_i}, i = 1, \dots, m$. Then, the system (1), and in turn system (5), are bilinearizable with an n -dimensional state space.*

Although the conditions of Theorem 1 may be hard to satisfy in a given problem, an approximation of the true system (1) can be obtained with sufficiently small approximation error by including adequately many eigenfunctions in the basis. As a result, $L_{g_i} = G_i$ and the system can be expressed as the *Koopman bilinear form* (KBF) (see [22] for details):

$$\dot{z} = Fz + \sum_{i=1}^m G_i z u_i, \quad z \in \mathbb{R}^n, n < \infty. \quad (6)$$

C. Nonlinear model predictive control

When the exact continuous dynamics (1) are known, the general optimal control problem is intractable because there are infinitely many optimization variables. To reformulate the problem into a tractable finite-dimensional nonlinear program (NLP) we discretize the dynamics. Given a time horizon T , consider the time increment Δt and divide the time horizon $[0, T]$ into $N = \frac{T}{\Delta t} + 1$ discrete subintervals $[t_k, t_{k+1}], t_k = k\Delta t, k = 0, \dots, N-1$. Replacing the continuous control signal $u(t)$ with a zero-order-hold signal, the dynamics are integrated over each interval with an appropriate integration scheme to get a discrete-time representation of the dynamics $x_{k+1} = f_d(x_k, u_k)$, where $x_k = x(t_k)$.

The quadratic objective NMPC problem is formulated as:

$$\begin{aligned} \min_{\mathbf{x}, \mathbf{u}} \quad & \sum_{k=0}^{N-1} \frac{1}{2} \begin{bmatrix} x_k - x_k^{\text{ref}} \\ u_k - u_k^{\text{ref}} \end{bmatrix}^T W_k \begin{bmatrix} x_k - x_k^{\text{ref}} \\ u_k - u_k^{\text{ref}} \end{bmatrix} \\ \text{s.t.} \quad & x_{k+1} = f_d(x_k, u_k), \quad k = 0, \dots, N-1 \\ & x_0 = \hat{x}, \quad h_k(x_k, u_k) \leq 0, \quad k = 0, \dots, N \end{aligned} \quad (7)$$

where $\hat{x}$ is the current state, $\mathbf{x} = [x_0, \dots, x_N]$, $\mathbf{u} = [u_0, \dots, u_{N-1}]$ are the stacked matrices of state and control vectors for each time step, W is the positive semi-definite cost matrix, and h_k is the constraint function encoding state and actuation constraints, both allowed to change at every timestep. In classical receding horizon fashion, at each timestep, a new state estimate $\hat{x}$ is obtained, the optimization problem is solved, and the control signal solution corresponding to the first timestep, u_0 , is deployed to the system.

D. Sequential quadratic programming

NMPC problems (7) are primarily solved via interior point (IP) or sequential quadratic programming (SQP) methods. SQP-approaches can leverage the fact that the nonlinear programming problems (NLP) solved at adjacent timesteps are quite similar, so that the solution of the NMPC problem

at the previous timestep can be used to *warm-start* the solution at the current timestep. This warm-start feature greatly reduces the real-time computational burden, and often a single SQP iteration is sufficient at each timestep to arrive at a close-to-optimal solution of the NMPC problem [1].

In the SQP algorithm, summarized in Algorithm 1, Eq. (7) is sequentially approximated by quadratic programs (QPs), whose solutions are Newton directions for performing steps toward the optimal solution of the NLP. The sequence is initialized at an initial guess of the solution, $(\mathbf{x}_0^{\text{init}}, \mathbf{u}_0^{\text{init}})$, at which the following QP is iteratively solved and the initial guess updated at each iteration i until convergence:

$$\begin{aligned} \min_{\Delta \mathbf{x}_i, \Delta \mathbf{u}_i} \quad & \sum_{k=0}^N \begin{bmatrix} \Delta \mathbf{x}_{i,k} \\ \Delta \mathbf{u}_{i,k} \end{bmatrix}^T H_{i,k} \begin{bmatrix} \Delta \mathbf{x}_{i,k} \\ \Delta \mathbf{u}_{i,k} \end{bmatrix} + J_{i,k}^T \begin{bmatrix} \Delta \mathbf{x}_{i,k} \\ \Delta \mathbf{u}_{i,k} \end{bmatrix} \\ \text{s.t.} \quad & \Delta \mathbf{x}_{i,k+1} = A_{i,k} \Delta \mathbf{x}_{i,k} + B_{i,k} \Delta \mathbf{u}_{i,k} + r_{i,k}, \quad k=0, \dots, N-1, \\ & C_{i,k} \Delta \mathbf{x}_{i,k} + D_{i,k} \Delta \mathbf{u}_{i,k} + h_{i,k} \leq 0, \quad k=0, \dots, N, \\ & \Delta \mathbf{x}_{i,0} = \hat{\mathbf{x}}_i - \mathbf{x}_{i,0}^{\text{init}}, \end{aligned} \quad (8)$$

where $H_{i,k}$ is the Hessian of the NLP Lagrangian (7) and

$$\begin{aligned} A_{i,k} &= \frac{\partial f_d}{\partial \mathbf{x}} \bigg|_{\mathbf{x}_{i,k}^{\text{init}}, \mathbf{u}_{i,k}^{\text{init}}}, \quad B_{i,k} = \frac{\partial f_d}{\partial \mathbf{u}} \bigg|_{\mathbf{x}_{i,k}^{\text{init}}, \mathbf{u}_{i,k}^{\text{init}}}, \quad C_{i,k} = \frac{\partial h}{\partial \mathbf{x}} \bigg|_{\mathbf{x}_{i,k}^{\text{init}}, \mathbf{u}_{i,k}^{\text{init}}}, \quad D_{i,k} = \frac{\partial h}{\partial \mathbf{u}} \bigg|_{\mathbf{x}_{i,k}^{\text{init}}, \mathbf{u}_{i,k}^{\text{init}}}, \\ r_{i,k} &= f_d(\mathbf{x}_{i,k}^{\text{init}}, \mathbf{u}_{i,k}^{\text{init}}) - \mathbf{x}_{i,k+1}^{\text{init}}, \quad h_{i,k} = h(\mathbf{x}_{i,k}^{\text{init}}, \mathbf{u}_{i,k}^{\text{init}}), \\ J_{i,k} &= W_{i,k} \begin{bmatrix} \mathbf{x}_{i,k}^{\text{init}} - \mathbf{x}_{i,k}^{\text{ref}} \\ \mathbf{u}_{i,k}^{\text{init}} - \mathbf{u}_{i,k}^{\text{ref}} \end{bmatrix}. \end{aligned} \quad (9)$$

III. LEARNING LIFTED BILINEAR DYNAMICS

A. Modeling assumptions and data collection

We use EDMD to learn approximate finite dimensional lifted bilinear dynamics from data. The system's unknown dynamics are assumed to be control-affine, with $f, g_1, \dots, g_m$ in (1) unknown. We seek to learn a model and design a multi-purpose controller for the unknown system.

We assume that a nominal controller permits us to execute M_t data collection trajectories of length T_t from initial conditions $\mathbf{x}_0^j \in \Omega, j = 1, \dots, M_t$. From each trajectory, $M_s = (T_t/\Delta t)$ state and control actions are sampled at a fixed time interval Δt , resulting in a data set

$$\mathcal{D} = \left(\left(x_k^j, u_k^j \right)_{k=0}^{M_s} \right)_{j=1}^{M_t}. \quad (10)$$

Algorithm 1: [2] SQP for NMPC at discrete time i

Input: current state $\hat{\mathbf{x}}_i$, reference trajectory $(\mathbf{x}_i^{\text{ref}}, \mathbf{u}_i^{\text{ref}})$, initial guess $(\mathbf{x}_i^{\text{init}}, \mathbf{u}_i^{\text{init}})$
while Not converged **do**
 Form $r_{i,k}, h_{i,k}, A_{i,k}, B_{i,k}, C_{i,k}, D_{i,k}, H_{i,k}, J_{i,k}$ by (9)
 Solve (8) to get the Newton direction $(\Delta \mathbf{x}_i, \Delta \mathbf{u}_i)$
 Update initial guess with the Newton step:
 $(\mathbf{x}_i^{\text{init}}, \mathbf{u}_i^{\text{init}}) \leftarrow (\mathbf{x}_i^{\text{init}} + \Delta \mathbf{x}_i, \mathbf{u}_i^{\text{init}} + \Delta \mathbf{u}_i)$
end
Return: NMPC solution $(\mathbf{x}_i, \mathbf{u}_i) = (\mathbf{x}_i^{\text{init}}, \mathbf{u}_i^{\text{init}})$

Since the NMPC design requires continuous-time models to be discretized, we learn a discrete-time lifted bilinear model, thereby avoiding potential numerical differentiation and discretization errors. This is further motivated by the existence of discretization procedures that maintain stability properties and the bilinear structure of the original system, such as the trapezoidal rule with zero-order-hold [26], [27].

B. Supervised learning of unknown dynamics

Define a dictionary of D dictionary functions $z = \phi(x), \phi : \mathbb{R}^d \rightarrow \mathbb{R}^D$. The choice of the functions can be based on system knowledge (i.e. feature engineering) or be a generic basis of functions such as monomials of the state up to a certain degree. Choosing dictionary functions is an ongoing area of research in Koopman-based learning methods. The most promising and principled choice of functions is arguably using data-driven Koopman eigenfunctions [28], [29]. However, existing methods cannot be readily applied in the KCT-setting and extending the methods to control-affine dynamics is outside of the scope of this paper.

To learn a lifted bilinear dynamic model, the data $\mathcal{D}$ is organized into data matrices X, X', U , where each corresponding column of X , and X' are state samples recorded one sampling interval apart, see (11). Then, the lifted data matrix is created by applying $\phi(x)$ to each column of X and X' , denoted $Z = \phi(X), Z' = \phi(X')$ by slight abuse of notation. Finally, Z_u is constructed by applying $\phi_u(x, u)$ to each corresponding pair of columns of X and U , where $\phi_u(x, u) = [\phi(x) \phi(x)u_1 \dots \phi(x)u_m]^T$. Learning can then be formulated as a linear regression problem (11).

$$\begin{aligned} \min_{F, G_1, \dots, G_m \in \mathbb{R}^{D \times D}} \quad & \|Z' - [F \quad G_1 \quad \dots \quad G_m] Z_u\|^2 \\ \min_{C^x \in \mathbb{R}^{d \times D}} \quad & \|X - C^x Z\|^2 \end{aligned} \quad (11)$$

$$\begin{aligned} X &= [x_0^1 \quad \dots \quad x_{M_s-1}^1 \quad \dots \quad x_0^{M_t} \quad \dots \quad x_{M_s-1}^{M_t}], \\ X' &= [x_1^1 \quad \dots \quad x_{M_s}^1 \quad \dots \quad x_1^{M_t} \quad \dots \quad x_{M_s}^{M_t}], \\ U &= [u_0^1 \quad \dots \quad u_{M_s-1}^1 \quad \dots \quad u_0^{M_t} \quad \dots \quad u_{M_s-1}^{M_t}], \\ Z &= \phi(X), \quad Z' = \phi(X'), \quad Z_u = \phi_u(X, U). \end{aligned}$$

Regularization, such as sparsity-promoting l_1 -regularization which has been shown to improve prediction performance and reduce overfitting [9], can be added to the regression. Furthermore, learning C^x is not needed if the projection from the lifted space to the original space can be analytically computed for the chosen dictionary. For example, a monomial basis will typically include the state itself. This results in a lifted discrete-time bilinear model of the form

$$x_k = C^x z_k, \quad z_{k+1} = F z_k + \sum_{l=1}^m G_l z_k u_{k,l}. \quad (12)$$

Algorithm 2: Koopman NMPC (closed loop)

Input: reference trajectory $(\mathbf{x}_i^{\text{ref}}, \mathbf{u}_i^{\text{ref}})$, initial guess $(\mathbf{x}_i^{\text{init}}, \mathbf{u}_i^{\text{init}})$
while Controller is running **do**
 Form $r_{i,k}, A_{i,k}, B_{i,k}$ using (15)
 Get and lift current state, $z_{i,0} = \phi(\hat{x})$
 Solve (13) to get the Newton direction $(\Delta \mathbf{x}_i, \Delta \mathbf{u}_i)$
 Update solution, $(\mathbf{x}_i, \mathbf{u}_i) \leftarrow (\mathbf{x}_i^{\text{init}} + \Delta \mathbf{x}_i, \mathbf{u}_i^{\text{init}} + \Delta \mathbf{u}_i)$
 Deploy first input u_0 to the system
 Construct $(\mathbf{x}_{i+1}^{\text{init}}, \mathbf{u}_{i+1}^{\text{init}})$ using (14)
end

IV. NONLINEAR MODEL PREDICTIVE CONTROL DESIGN

A. Design considerations

Based on Section II, we first reformulate the NMPC problem (7) using the identified Koopman bilinear model:

$$\begin{aligned} \min_{Z, U} \quad & \sum_{i=0}^N \left[\mathbf{z}_{i,k}^{\text{init}} + \Delta \mathbf{z}_{i,k} \right]^T W_{i,k} \left[\mathbf{z}_{i,k}^{\text{init}} + \Delta \mathbf{z}_{i,k} \right] \\ \text{s.t.} \quad & z_{k+1} = Fz_k + \sum_{i=1}^m G_i z_k u_k^{(i)}, \quad k = 0, \dots, N-1 \\ & c_l \leq C^x z_k \leq c_u, \quad d_l \leq u_k \leq d_u, \quad k = 0, \dots, N, \\ & z_0 = \phi(\hat{x}). \end{aligned} \quad (13)$$

The initialization and closed loop operation of the controller can be summarized as follows (see Algorithm 2). Before task execution, the SQP algorithm with the Koopman QP subproblem (13) is executed to convergence to obtain a good initial guess of the solution. Then, in closed loop operation, the Koopman bilinear model is linearized along the initial guess, the current state is obtained from the system, the current state is lifted using the function basis, and then the QP subproblem is solved only once. Finally, the first control input of the optimal control sequence is deployed to the system, and the full solution is shifted one timestep and used as an initial guess at the next timestep.

Although we have restricted the objective to be quadratic and the state and actuation constraints to be linear (except for the evolution of the dynamics), nonlinear objective and constraint terms can be included by adding them to the lifted state $z = \phi(x)$. For example, if it is desired to enforce the constraint $\cos(x_1) \leq 0$, we can add $\phi_j = \cos(x_1)$ to the lifted state and enforce $z_k^{(j)} \leq 0$ [14].

While not our main focus, a discussion of how to achieve guaranteed closed loop stability of the proposed control strategy is in order. In the nominal case, with no model mismatch between the true dynamics and the Koopman bilinear model, closed loop stability of the controller for bilinear systems with a quasi-infinite method has been shown (see e.g. [30]). More recently, some early stability results using Lyapunov MPC methods have been developed [25]. In particular, the bilinear model structure simplifies the construction of a Lyapunov function that is added as a constraint to the MPC. Lyapunov stability of the controller based on the KBF is then proved under the assumption that the prediction error of the learned Koopman model is finite. Although promising,

further analysis of robustness and stability properties of the methodology is needed. In this work however, we focus on the practical implementation and defer further theoretical development to future research.

B. Warm-start of SQP at each timestep

As discussed in Section II-D, the SQP algorithm requires an initial guess of the solution $\mathbf{x}_i^{\text{init}}, \mathbf{u}_i^{\text{init}}$. Selecting a initial guess that is sufficiently close to the true optimal solution is essential for the algorithm to converge fast and reliably [2]. It is well known that the receding horizon nature of MPC can be exploited to obtain excellent initial guesses. At a time instant i , this can be achieved by *shifting* the NMPC solution from the previous timestep $i-1$ and by updating the guess of the final control input. Under certain conditions, a locally stable controller enforcing state and actuation constraints can be designed allowing feasibility of the initial guess to be guaranteed [31]. Typically, simpler approaches are taken such as simply adding a copy of the final control signal and calculating the implied final state using the dynamics model

$$\begin{aligned} \mathbf{u}_{i,k}^{\text{init}} &= \mathbf{u}_{i-1,k+1}, \quad k = 0, \dots, N-2, \\ \mathbf{x}_{i,k}^{\text{init}} &= \mathbf{x}_{i-1,k+1}, \quad k = 0, \dots, N-1, \\ \mathbf{u}_{i,N-1}^{\text{init}} &= \mathbf{u}_{i,N-2}^{\text{init}}, \quad \mathbf{x}_{i,N}^{\text{init}} = f_d(\mathbf{x}_{i,N-1}^{\text{init}}, \mathbf{u}_{i,N-1}^{\text{init}}). \end{aligned} \quad (14)$$

If the previous solution $\mathbf{x}_{i-1}, \mathbf{u}_{i-1}$ is feasible, the shifted solution will also be feasible for all but the last timestep.

C. Calculating the linearized system matrices

As a result of the bilinear structure of the dynamics model, the linearization can be efficiently computed for a given initial guess. The linearization at each timestep $k = 0, \dots, N-1$ of the initial guess is obtained by directly calculating the partial derivatives as described in (9)

$$\begin{aligned} A_{i,k} &= F + \sum_{j=1}^m G_j (\mathbf{u}_{i,k}^{\text{init}})^{(j)}, \quad B_{i,k} = [G_1 \mathbf{z}_{i,k}^{\text{init}} \dots G_m \mathbf{z}_{i,k}^{\text{init}}] \\ r_{i,k} &= F \mathbf{z}_{i,k}^{\text{init}} + \sum_{j=1}^m G_j \mathbf{z}_{i,k}^{\text{init}} (\mathbf{u}_{i,k}^{\text{init}})^{(j)} - \mathbf{z}_{i,k+1}^{\text{init}}. \end{aligned} \quad (15)$$

Consequently, the linearized dynamics matrices can be obtained by simple matrix multiplication and addition with the dynamics matrices of the Koopman model and the matrices containing the initial guesses of $\mathbf{z}_i^{\text{init}}, \mathbf{u}_i^{\text{init}}$.

V. SIMULATED QUADROTOR LEARNING AND CONTROL

A. System and data collection details

Consider a planar quadrotor with states $\mathbf{x} = [y \ z \ \dot{y} \ \dot{z} \ \dot{\theta}]^T$,

$$\begin{bmatrix} \ddot{y} \\ \ddot{z} \\ \ddot{\theta} \end{bmatrix} = \begin{bmatrix} 0 \\ -g \\ 0 \end{bmatrix} + \begin{bmatrix} -(1/m)\sin\theta & -(1/m)\sin\theta \\ (1/m)\cos\theta & (1/m)\cos\theta \\ -l_{arm}/I_{xx} & l_{arm}/I_{xx} \end{bmatrix} \begin{bmatrix} T_1 \\ T_2 \end{bmatrix}, \quad (16)$$

where y, z describe the horizontal and vertical position in a fixed reference frame, θ is the orientation, T_1, T_2 are the propeller thrusts, g is the acceleration of gravity, m is the

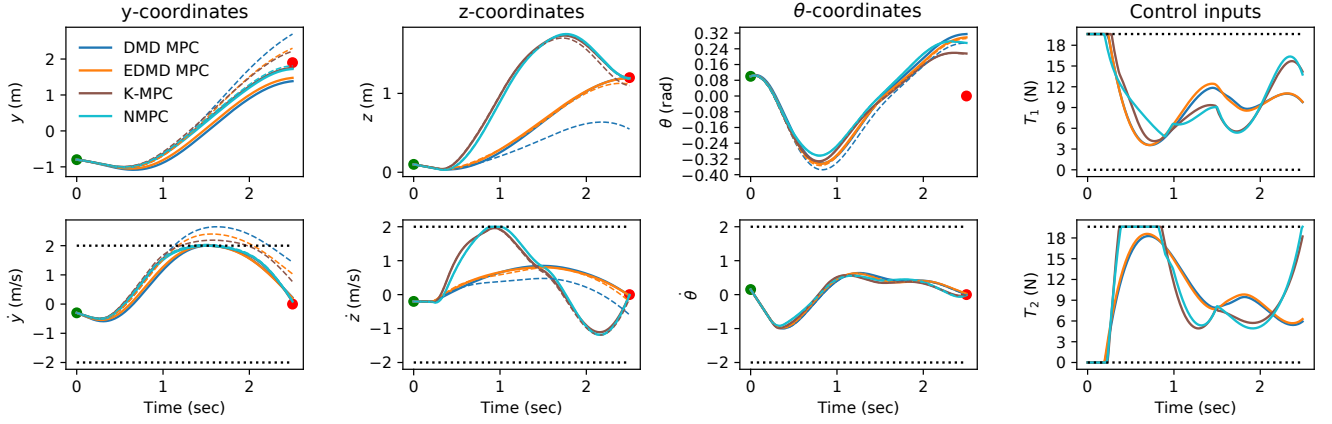


Fig. 1: Trajectories generated with MPCs based on DMD, EDMD, and bEDMD models. True model-based NMPC used as benchmark. (Black dotted lines - state/actuation constraints, dashed lines - open loop simulation of generated trajectories).

vehicle mass, l_{arm} is the distance from the vehicle's center of mass to the propeller axis, and I_{xx} is the rotational inertia.

To collect data, a nominal LQR controller is designed for the linearized dynamics at hover. Since the system is underactuated, learning trajectories are generated from a MPC based on the linearized dynamics. However, any controller can be used and the method does not need a known model linearization. Additionally, exploratory Gaussian white noise is added to aid sufficient excitation. The learning data set is collected as follows. First, an initial condition x_0 and final condition x_f , are sampled uniformly at random from the interval $y, z \in [-2, 2]^2, \theta \in [-\pi/3, \pi/3], \dot{y}, \dot{z}, \dot{\theta} \in [-1, 1]^3$. Then, 2-second long trajectories link x_0 to x_f , which are tracked via the LQR-controller. This process is repeated 100 times as state and actuation data is captured at 100 hz.

To compare our method against the state-of-the art of Koopman-based learning methods, we trained three separate models, dynamic mode decomposition (DMD) [32], extended DMD (EDMD) [33], and the method of Section III, denoted bilinear EDMD (bEDMD). Assuming that the input forces enter through rotation matrices, we chose a simple dictionary of 27 functions for both the EDMD and bEDMD consisting of the state vector and monomials of the $\theta, \dot{\theta}$ up to the third order multiplied by $1, \cos(\theta), \sin(\theta), \phi(x) = [1, y, z, \theta, \dot{y}, \dot{z}, \dot{\theta}, \theta^2, \theta, \dot{\theta}, \dots, \theta^3, \cos \theta]^T$. l_1 -regularization tuned with cross-validation was also applied to each method. Code for learning and control is implemented in Python (github.com/Cafolkes/koopman-learning-and-control) and the dynamics are simulated using 5th order Runge-Kutta in *scipy*.

B. Open loop prediction

We first evaluate our method's prediction performance. A test data set is generated the same way as the training set. Then, the control sequence of each test trajectory is simulated forward with each of the learned models. The mean and standard deviation of the error between the true and predicted evolution over the trajectories are reported in Table I. The experimental results support the theory: the

	DMD	EDMD	bEDMD
Mean squared error	8.71e-2	5.60e-2	7.53e-3
Improvement vs DMD		35.75 %	91.35 %
Improvement vs EDMD			86.54 %
Standard deviation	2.79e-1	2.36e-1	8.66e-2
Improvement vs DMD		15.27 %	68.94 %
Improvement vs EDMD			63.35 %

TABLE I: Prediction error of DMD, EDMD, and bEDMD models.

mean and standard deviation of the error is reduced by 86-91 percent and 63 to 69 percent, respectively, compared to DMD and EDMD. bEDMD better captures the nonlinearities in the actuation matrix that drives the (y, z) -dynamics.

C. Trajectory generation and closed loop control

To study trajectory generation and control, we first design MPCs for each of the learned models. For the linear (DMD) and lifted linear (EDMD) models, a linear MPC is designed. Then, the Koopman NMPC (K-NMPC) is designed as described in Section IV. Finally, as a benchmark we implement NMPC using the true dynamics (16) based on Section II-C. Each controller is based on a discrete-time model with sampling length 10 ms. All the optimization problems are solved with *OSQP* [34]. We initially study the ability of each controller to generate high quality trajectories. One hundred trajectories (2.5 second duration) are designed to move the system from x_0 to x_f , sampled uniformly at random from the interval $y, z \in [-2, 2]^2, \theta \in [-0.1, 0.1], \dot{y}, \dot{z}, \dot{\theta} \in [-1, 1]^3$. The Frobenius norm of the control inputs over the prediction horizon is minimized and a terminal state constraint is added to each of the controllers to require that the desired position is reached. Finally, the velocities are constrained to have magnitude less than 2, $\dot{y}, \dot{z}, \dot{\theta} \in [-2, 2]$, and the thrust of each propeller is limited, $T_1, T_2 \in [0, 2T_{hover}]$.

The generated trajectories from one of the experiments (solid lines) along with the open-loop simulation of the true dynamics with the control sequence of each designed trajectory (dashed lines) are depicted in Figure 1. Table II presents summary statistics from 100 experiments: total control effort (as measured by the Frobenius norm and normalized by the NMPC control effort), the terminal state error (the Euclidean distance between the final open loop

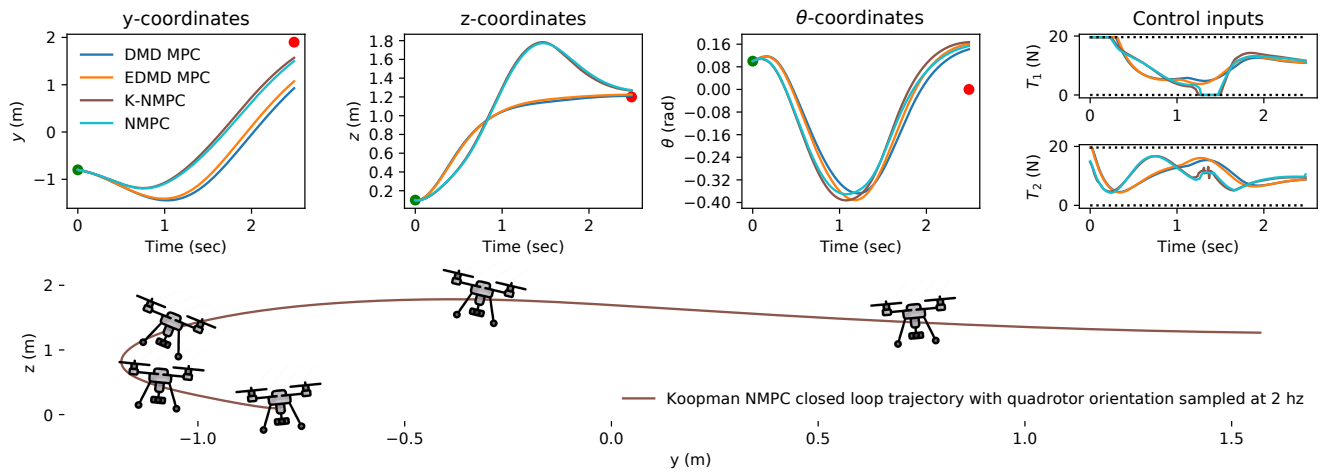


Fig. 2: Closed loop control with MPCs based on DMD, EDMD, and bEDMD models. True model-based NMPC used as benchmark.

stimulation state and the desired state), and the number of SQP iterations needed by K-NMPC and NMPC. The open loop simulation reveals that the trajectories resulting from the DMD and EDMD models are not realizable, leading to significant mean terminal state errors of 2.24 and 2.46, respectively. K-NMPC has a significantly lower mean error of 0.70, and, more importantly, captures the idealized behavior of NMPC, even though it is completely data-driven.

Finally, we study closed loop control behavior of each control approach over the same 100 initial and terminal conditions. Each of the MPCs use a 0.5 second prediction horizon with sampling length 10 ms and a quadratic state penalty and control input cost. The Koopman NMPC and NMPC controllers are initialized by solving each of the NLPs to convergence with the SQP algorithm before only a single SQP iteration is performed at each timestep in closed loop.

The traces resulting from each controller are presented in Figure 2 for one of the experiments. Furthermore, summary statistics over the 100 experiments of the realized cost (as measured by the total trajectory cost, normalized by the NMPC cost), and the computation time at each timestep of each of the controllers are reported in Table III. Because closed loop operation can correct for model errors, the performance difference between the controllers is smaller than for the trajectory generation case. The controllers based on the DMD and EDMD models achieve a 4 and 2 percent higher cost than the NMPC, respectively. The K-NMPC again closely follows the behaviour of the NMPC.

The linear and lifted linear MPCs require less computational effort than the SQP-based approaches with an average computation time of 2 and 7 ms, respectively. K-NMPC requires somewhat higher computational effort than NMPC

for this system with an average of 13 ms compared to 7 ms. This is dominated by longer solution time of the QP because a higher number of variables and constraints as a result of the lifting. The relative computational effort between K-NMPC and NMPC will ultimately depend on the complexity of linearizing the nonlinear model for NMPC, which can be expensive for complicated models, and the lifting dimension of the Koopman bilinear model. Finally, we note that even with a relatively simple python implementation the controllers are approaching real-time capability.

VI. CONCLUSION

This paper presented a method to combine the learning of lifted bilinear models based on Koopman spectral theory with nonlinear model predictive control. This combination enables flexible data-driven control design with the many advantages of NMPC, such as state and actuation constraint satisfaction and optimality with respect to intuitive performance specifications. Through a simulated planar quadrotor example, we demonstrated the advantages of learning lifted bilinear models over lifted linear models to capture control-affine dynamics. We also showed that the resulting data-driven controller could achieve similar performance to the case of NMPC design with an exactly known dynamic model.

Future work includes data-driven learning of the dictionary functions in the lifted bilinear model setting and applying the method experimentally to a multirotor drone. Learned dictionary functions will enable similar or better performance with more compact models (i.e. fewer functions in the dictionary). As a result, higher-dimensional system dynamics can be learned with the method while maintaining acceptable computation times for the K-NMPC.

	DMD (MPC)		EDMD (MPC)		bEDMD (K-NMPC)		Benchmark (NMPC)	
	mean	std	mean	std	mean	std	mean	std
Control effort	0.97	0.02	0.97	0.06	1.01	0.01	1.00	0.00
Terminal error	2.24	1.50	2.46	1.77	0.70	0.31	0.36	0.17
SQP iterations					21.88	11.57	9.33	9.63

TABLE II: Summary statistics over 100 experiments of the MPC trajectory cost, error, and SQP iterations.

	DMD (MPC)		EDMD (MPC)		bEDMD (K-NMPC)		Benchmark (NMPC)	
	mean	std	mean	std	mean	std	mean	std
Realized cost	1.04	0.05	1.02	0.03	1.00	0.00	1.00	0.00
Comp time (ms)	1.50	0.34	7.40	1.78	13.14	8.35	6.99	1.11

TABLE III: Summary statistics over 100 experiments of the MPC closed loop control cost and computation times.

REFERENCES

- [1] D. Kouzoupis, G. Frison, A. Zanelli, and M. Diehl, "Recent Advances in Quadratic Programming Algorithms for Nonlinear Model Predictive Control," *Vietnam Journal of Mathematics*, vol. 46, no. 4, pp. 863–882, 2018.
- [2] S. Gros, M. Zanon, R. Quirynen, A. Bemporad, and M. Diehl, "From linear to nonlinear MPC: bridging the gap via the real-time iteration," *International Journal of Control*, vol. 93, no. 1, pp. 62–80, 2020.
- [3] R. Grandia, A. J. Taylor, A. Singletary, M. Hutter, and A. D. Ames, "Nonlinear Model Predictive Control of Robotic Systems with Control Lyapunov Functions," 2020.
- [4] S. Lupashin, M. Hehn, M. W. Mueller, A. P. Schoellig, M. Sherback, and R. D'Andrea, "A platform for aerial robotics research and demonstration: The Flying Machine Arena," *Mechatronics*, 2014.
- [5] J. M. Wang, D. J. Fleet, and A. Hertzmann, "Gaussian Process Dynamical Systems," in *Proc. Neural Information Processing Systems*, 2006.
- [6] G. Shi, X. Shi, M. O'Connell, R. Yu, K. Azizzadenesheli, A. Anandkumar, Y. Yue, and S.-J. Chung, "Neural Lander: Stable Drone Landing Control using Learned Dynamics," *International Conference on Robotics and Automation (ICRA)*, pp. 9784–9790, 2019.
- [7] A. J. Taylor, V. D. Dorobantu, H. M. Le, Y. Yue, and A. D. Ames, "Episodic Learning with Control Lyapunov Functions for Uncertain Robotic Systems," 2019.
- [8] B. Recht, "A Tour of Reinforcement Learning: The View from Continuous Control," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 2, no. 1, pp. 253–279, 2019.
- [9] E. Kaiser, J. N. Kutz, and S. L. Brunton, "Sparse identification of nonlinear dynamics for model predictive control in the low-data limit," *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, vol. 474, no. 2219, 2018.
- [10] Y. Lan and I. Mezić, "Linearization in the large of nonlinear systems and Koopman operator spectrum," *Physica D*, vol. 242, no. 1, pp. 42–53, 2013.
- [11] A. Mauroy and J. Goncalves, "Linear identification of nonlinear systems: A lifting technique based on the Koopman operator," in *2016 IEEE 55th Conference on Decision and Control (CDC)*, pp. 6500–6505, IEEE, 2016.
- [12] P. J. Schmid, "Dynamic mode decomposition of numerical and experimental data," *Journal of Fluid Mechanics*, vol. 656, pp. 5–28, 2010.
- [13] M. O. Williams, I. G. Kevrekidis, and C. W. Rowley, "A Data-Driven Approximation of the Koopman Operator: Extending Dynamic Mode Decomposition," *Journal of Nonlinear Science*, vol. 25, no. 6, pp. 1307–1346, 2015.
- [14] M. Korda and I. Mezić, "Linear predictors for nonlinear dynamical systems: Koopman operator meets model predictive control," *Automatica*, vol. 93, pp. 149–160, 2018.
- [15] K. Taira, S. L. Brunton, S. T. M. Dawson, C. W. Rowley, T. Colonius, B. J. McKeon, O. T. Schmidt, S. Gordyev, V. Theofilis, and L. S. Ukeiley, "Modal Analysis of Fluid Flows: An Overview," *AIAA Journal*, vol. 55, no. 12, pp. 4013–4041, 2017.
- [16] E. Berger, M. Sastuba, D. Vogt, B. Jung, and H. Ben Amor, "Estimation of perturbations in robotic behavior using dynamic mode decomposition," *Advanced Robotics*, vol. 29, pp. 331–343, 3 2015.
- [17] D. Goswami and D. A. Paley, "Global bilinearization and controllability of control-affine nonlinear systems: A Koopman spectral approach," *2017 IEEE 56th Annual Conference on Decision and Control, CDC 2017*, vol. 2018-Janua, no. Cdc, pp. 6107–6112, 2018.
- [18] B. W. Brunton, L. A. Johnson, J. G. Ojemann, and J. N. Kutz, "Extracting spatial-temporal coherent patterns in large-scale neural recordings using dynamic mode decomposition," *Journal of Neuroscience Methods*, vol. 258, pp. 1–15, 2016.
- [19] J. L. Proctor, S. L. Brunton, and J. Nathan Kutz, "Generalizing Koopman Theory to Allow for Inputs and Control *," vol. 17, no. 1, pp. 909–930, 2018.
- [20] A. Surana, "Koopman operator based observer synthesis for control-affine nonlinear systems," *2016 IEEE 55th Conference on Decision and Control, CDC 2016*, no. Cdc, pp. 6492–6499, 2016.
- [21] C. Folkestad, D. Pastor, and J. W. Burdick, "Episodic Koopman Learning of Nonlinear Robot Dynamics with Application to Fast Multirotor Landing," in *Proc. International Conference of Robotics and Autonomy*, 2020.
- [22] D. Bruder, B. Gillespie, C. David Remy, and R. Vasudevan, "Modeling and Control of Soft Robots Using the Koopman Operator and Model Predictive Control," 2019.
- [23] B. Huang, X. Ma, and U. Vaidya, "Feedback Stabilization Using Koopman Operator," *Proceedings of the IEEE Conference on Decision and Control*, vol. 2018-Decem, no. 1, pp. 6434–6439, 2019.
- [24] D. Bruder, X. Fu, and R. Vasudevan, "Advantages of Bilinear Koopman Realizations for the Modeling and Control of Systems with Unknown Dynamics," 2020.
- [25] A. Narasingam and J. S.-I. Kwon, "Data-driven feedback stabilization of nonlinear systems: Koopman-based model predictive control," pp. 1–11, 2020.
- [26] M. Q. Phan, Y. Shi, R. Betti, and R. W. Longman, "Discrete-time bilinear representation of continuous-time bilinear state-space models," in *Advances in the Astronautical Sciences*, 2012.
- [27] A. Surana, M. O. Williams, M. Morari, and A. Banaszuk, "Koopman operator framework for constrained state estimation," in *2017 IEEE 56th Annual Conference on Decision and Control, CDC 2017*, 2018.
- [28] C. Folkestad, D. Pastor, I. Mezic, R. Mohr, M. Fonoberova, and J. Burdick, "Extended Dynamic Mode Decomposition with Learned Koopman Eigenfunctions for Prediction and Control," in *Proc. American Control Conference*, 2019.
- [29] M. Korda and I. Mezic, "Optimal construction of Koopman eigenfunctions for prediction and control," *IEEE Transactions on Automatic Control*, pp. 1–1, 2020.
- [30] H. H. Bloemen, M. Cannon, and B. Kouvaritakis, "Closed-loop stabilizing MPC for discrete-time bilinear systems," *European Journal of Control*, vol. 8, no. 4, pp. 304–314, 2002.
- [31] J. B. Rawlings and D. Q. Mayne, "Model Predictive Control Theory and Design," 2012.
- [32] J. H. Tu, C. W. Rowley, D. M. Luchtenburg, S. L. Brunton, and J. N. Kutz, "On dynamic mode decomposition: Theory and applications," *Journal of Computational Dynamics*, vol. 1, no. 2, pp. 391–421, 2014.
- [33] Q. Li, F. Dietrich, E. M. Bollt, and I. G. Kevrekidis, "Extended dynamic mode decomposition with dictionary learning: A data-driven adaptive spectral decomposition of the koopman operator," *Chaos*, vol. 27, no. 10, 2017.
- [34] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd, "OSQP: An Operator Splitting Solver for Quadratic Programs," *2018 UKACC 12th International Conference on Control, CONTROL 2018*, no. 1, p. 339, 2018.